Programação concorrente

1º semestre de 2026

Sincronização

Tema #05

Professor Marcelo Eustáquio

Concorrência trata da coordenação de tarefas simultâneas; e distribuição trata da coordenação de sistemas independentes.

Marcelo Eustáquio

Programação concorrente

Refere-se à execução de múltiplas tarefas aparentemente ao mesmo tempo dentro de um *mesmo sistema computacional*. Normalmente, programação concorrente pode ocorrer em múltiplas threads, em múltiplos processos, em um **único computador** e compartilhando memória.

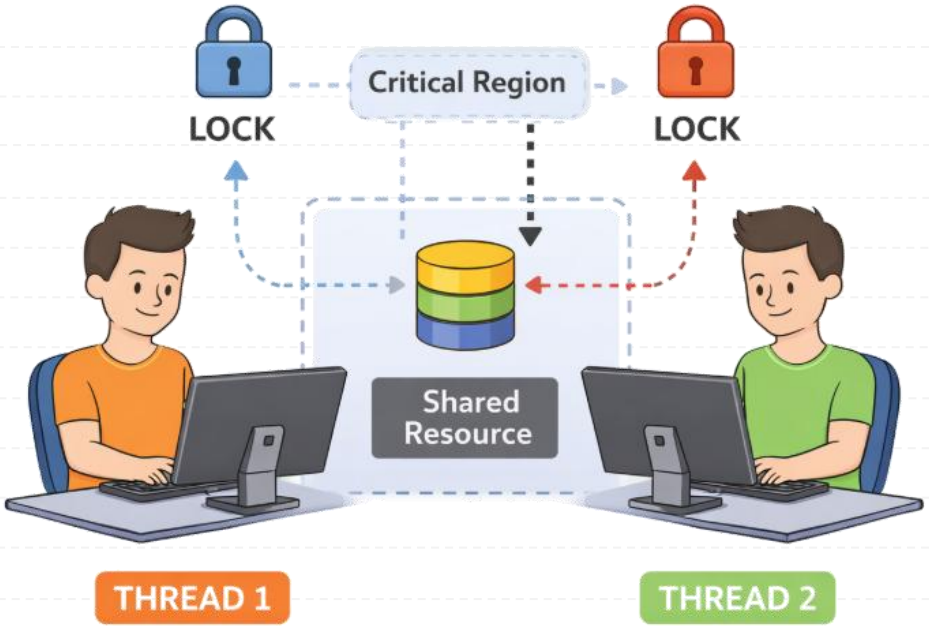
O principal foco da **programação concorrente** está na *coordenação entre tarefas simultâneas*.

São características da programação concorrente:

1	Compartilhamento de memória.
2	Sincronização com mutex (locks), semáforos e monitores.
3	Problemas típicos (condições de corrida, deadlock e starvation).

Programação concorrente

Programação concorrente pode ser ilustrada no funcionamento do sistema operacional, em programas multithread ou em servidores web multithread.



Programação distribuída

Diferentemente da programação concorrente, a programação distribuída refere-se à execução de tarefas em *múltiplos computadores conectados em rede.*

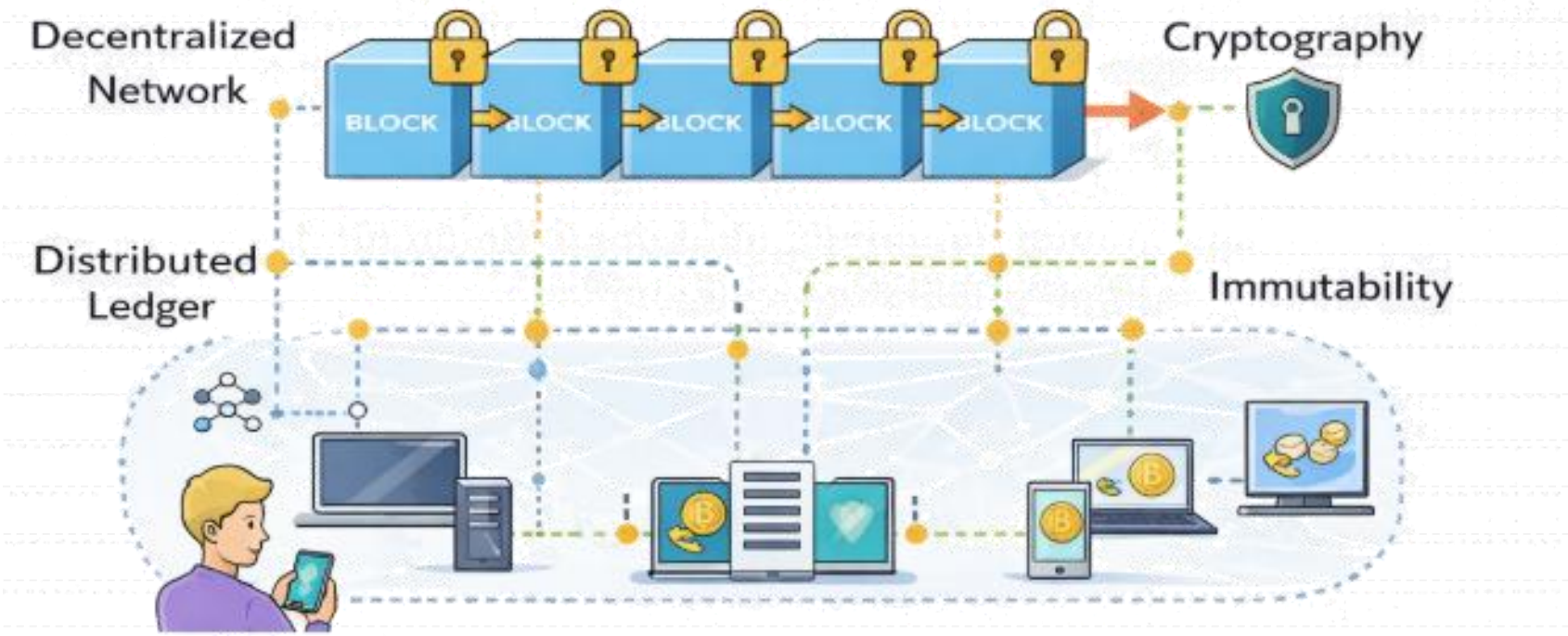
O principal foco da **programação distribuída** está na *coordenação entre nós independentes.*

São características da programação concorrente:

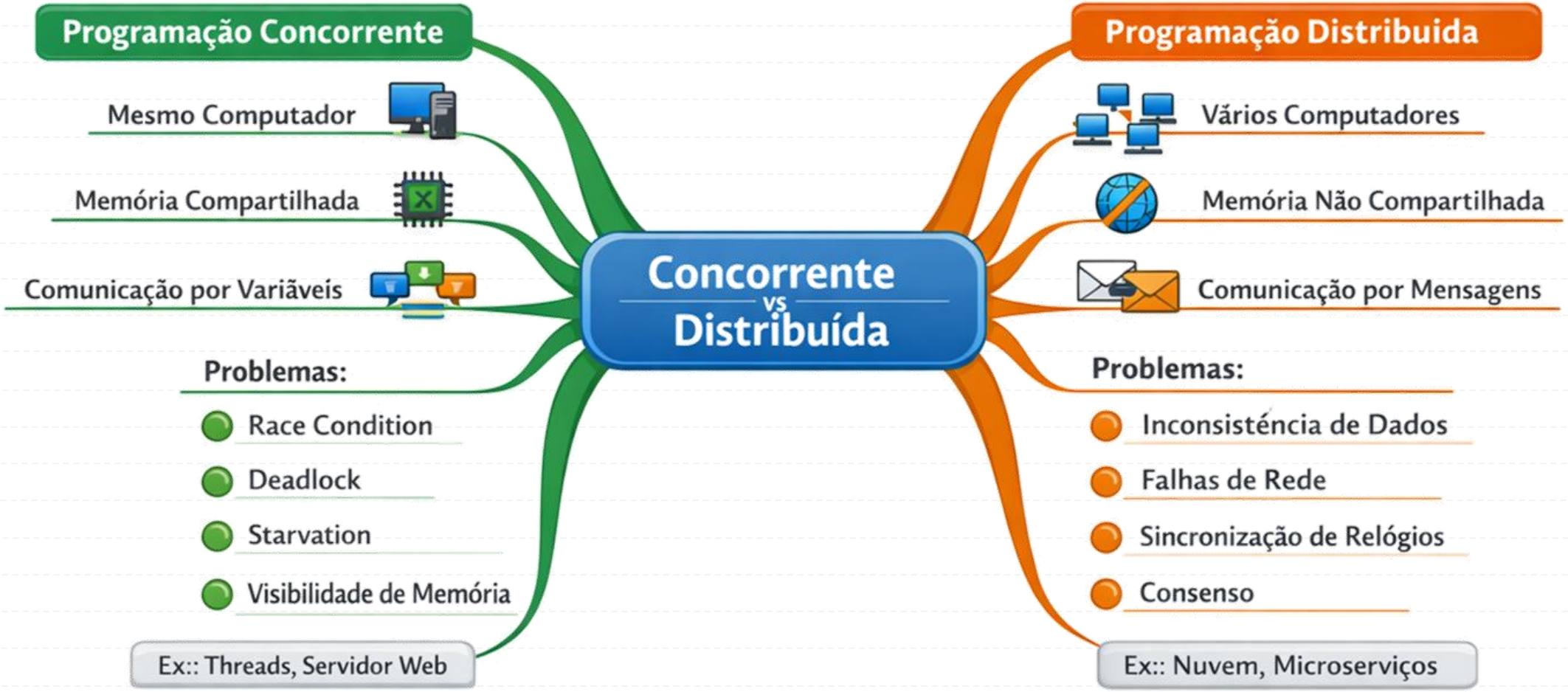
1	O não compartilhamento de memória.
2	Comunicação por mensagens (podem ser usados XML e JSON, por exemplo).
3	Presença de latência de rede e possibilidade de falhas parciais.

Programação distribuída

São problemas típicos da programação distribuída as inconsistências de dados, as falhas de rede, a falta de sincronização de relógios, a replicação, o consenso e o particionamento. Ademais, programação distribuída pode ser observada em sistemas bancários, sistemas de nuvem, microserviços e blockchain.



Esquema de funcionamento do blockchain.



Concorrente: Coordenação de Tarefas

Distribuída: Coordenação de Sistemas

xxx

Marcelo Eustáquio

Condição de corrida

Condição de corrida (race condition) é um problema que ocorre em programação concorrente ou paralela quando duas ou mais threads/processos acessam e manipulam dados compartilhados ao mesmo tempo, e o resultado final depende da ordem imprevisível de execução dessas operações.

Ou seja:

1	O sistema funciona corretamente apenas se as operações ocorrerem em uma certa ordem.
2	Como a ordem real pode variar, o resultado pode ser incorreto ou inconsistente.

Importante:

*Condição de corrida ocorre quando o resultado da execução depende da ordem de **interleaving** (ordem em que as instruções de diferentes threads/processos são intercaladas durante a execução concorrente).*

Atenção:

Uma condição de corrida acontece quando duas ou mais threads acessam e modificam um mesmo dado ao mesmo tempo, sem controle — gerando resultados imprevisíveis.

```python
import threading

contador = 0

def incrementar():
    global contador
    for _ in range(100000): contador += 1

threads = []

for _ in range(2):  # duas threads
    t = threading.Thread(target=incrementar)
    threads.append(t)
    t.start()

for t in threads: t.join()

print("Valor final:", contador)
```

Matematicamente, o valor esperado seria 200000. No entanto, muitas vezes o resultado será *menor*, porque as threads "*brigam*" pelo acesso à variável contador. Isso acontece a operação **contador += 1** não é atômica.

Matematicamente, o valor esperado seria 200000. No entanto, muitas vezes o resultado será *menor*, porque as threads "*brigam*" pelo acesso à variável contador. Isso acontece a operação **contador += 1** não é atômica.

Uma operação é **não atômica** quando pode ser desdobrada em operações mais simples, como é o caso de contador += 1 que, internamente, faz a *leitura* das variável global contador (1ª operação); *adiciona* uma unidade (2ª operação) e *escreve* o valor adicionado em contador (3ª operação). Assim, se duas threads fizerem isso ao mesmo tempo, uma atualização pode ser perdida.

Uma das possibilidades de se evitar o problema de condições de corrida está no uso de mecanismos de bloqueio como **Mutex** (implementado via Lock's).

```python
import threading

contador = 0
lock = threading.Lock()

def incrementar():
    global contador
    for _ in range(100000):

        with lock:
            contador += 1

threads = []

for _ in range(2):
    t = threading.Thread(target=incrementar)
    threads.append(t)
    t.start()

for t in threads: t.join()

print("Valor final:", contador)
```

Mutex

O **Lock** (*trava* / *mutex*) é um mecanismo de **sincronização** usado para controlar o acesso concorrente a um recurso compartilhado (variável, arquivo, memória etc.), garantindo que apenas uma thread por vez execute um trecho crítico do código.

```
contador = 0
def incrementar():
    global contador
    for _ in range(100000): contador += 1
```

Quando duas threads fazem **contador += 1** pode acontecer:

1) *Thread A lê contador = 10*
2) *Thread B lê contador = 10*
3) *Thread A grava 11*
4) *Thread B grava 11*

O resultado correto seria **12**, mas ficou **11** (**condição de corrida**).

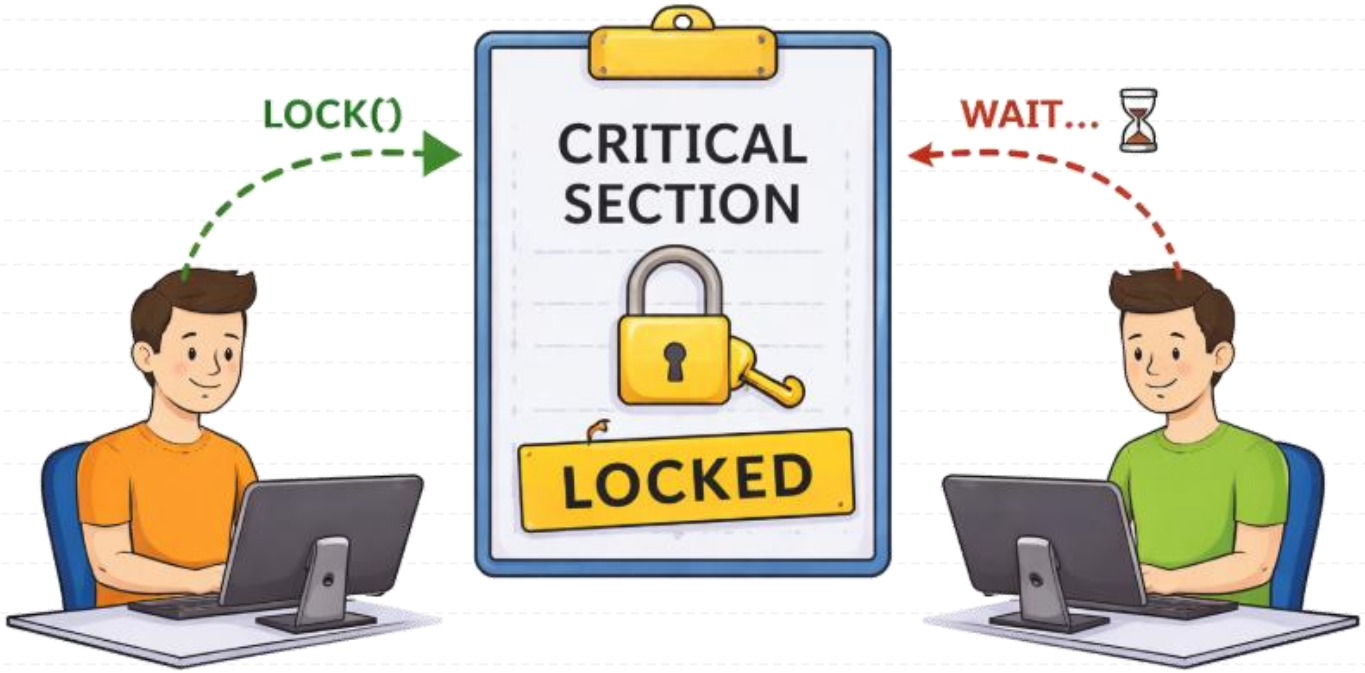
Mutex

O **Lock** (*trava/mutex*) é um mecanismo de **sincronização** usado para controlar o acesso concorrente a um recurso compartilhado, garantindo que apenas uma thread por vez execute um trecho crítico do código.

```
contador = 0
lock = threading.Lock()
def incrementar():
    global contador
    for _ in range(100000):
        with lock:
            contador += 1
```

O lock cria uma fila de acesso exclusivo, desenvolvendo o seguinte fluxo:

1) *Thread A pede o lock*
2) *Lock libera acesso e A acessa a região crítica*
3) *Thread B tenta entrar (fica bloqueada)*
4) *Thread A termina e libera lock*
5) *Thread B entra*



Mutex é o trecho protegido com ***with lock*** e significa que “somente uma thread pode executar isso por vez”, garantindo exclusão mútua, evitando corrupção de dados, tornando a execução determinística e sincronizando threads.

Mutex

Um **Lock** tem basicamente dois estados (**livre** e **ocupado**) que utilizam, basicamente, de duas operações:

- **acquire()**: tenta “pegar” o lock para acessar a área compartilhada.
- **release()**: libera o lock.

O with lock faz isso automaticamente:

```
contador = 0
lock = threading.Lock()
def incrementar():
    global contador
    for _ in range(100000):
        with lock:
            contador += 1
```

Equivalente:

lock.acquire()
contador += 1
lock.release()

Mutex

No gerenciamento de processos e *threads*, as variáveis mutexes são usadas para comunicação entre processos por meio de primitivas como *send* e *receive*.

Um semáforo é um mecanismo de sincronização usado em programação concorrente para controlar o acesso de múltiplas threads a recursos compartilhados por meio de um contador e operações atômicas.

Marcelo Eustáquio

Semáforo

Semáforos são mecanismos de **sincronização** usados em programação concorrente para controlar o acesso a múltiplas threads ou recursos compartilhados.

Em termos simples:

São estruturas que funcionam como um "**contador com regras**" que decidem que pode entrar e quem deve esperar para se ter acesso à área protegida.

Ou sema, semáforo é uma estrutura similar e mais geral que um mutex que mantém um valor interno inteiro. Por esse motivo, pode ser classificado em:

1	Semáforos binários, que armazena um dos valores 0 e 1, e funciona como um mutex simples.
2	Semáforos contadores, que controla vários recursos idênticos e guarda valores como 0, 1, 2, 3, ...

Semáforo

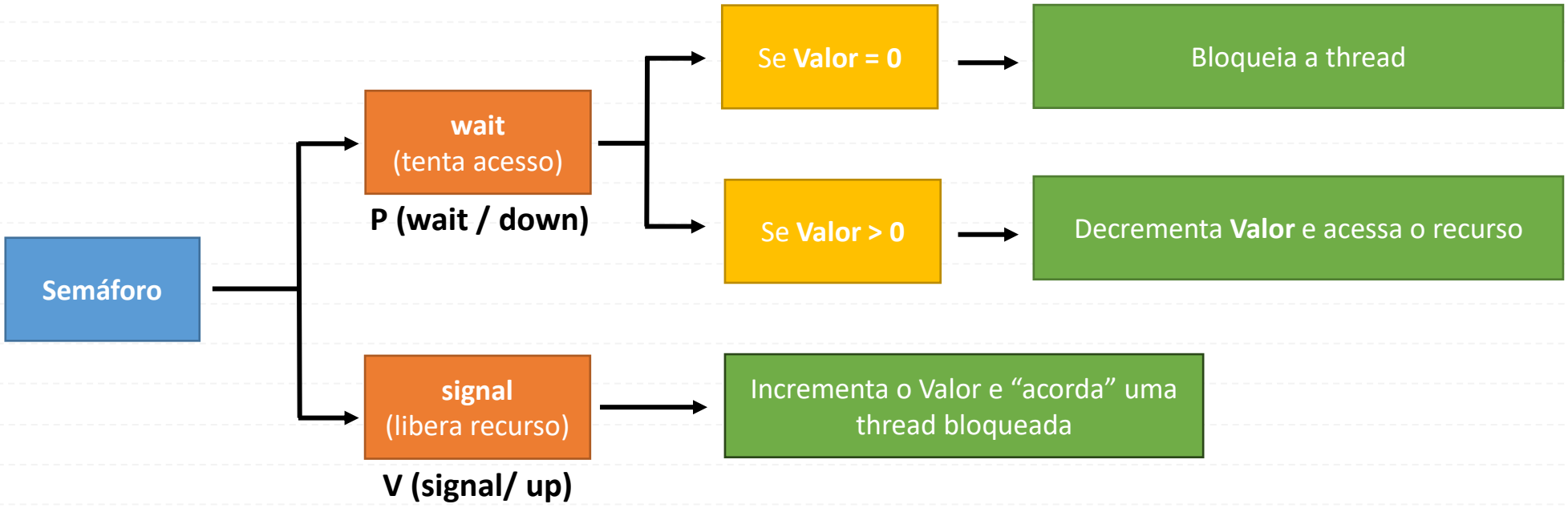
O conceito de semáforo, enquanto mecanismo de sincronização e controle de acesso a recursos compartilhados, foi proposto por **Edsger Dijkstra** e é fundamental para:

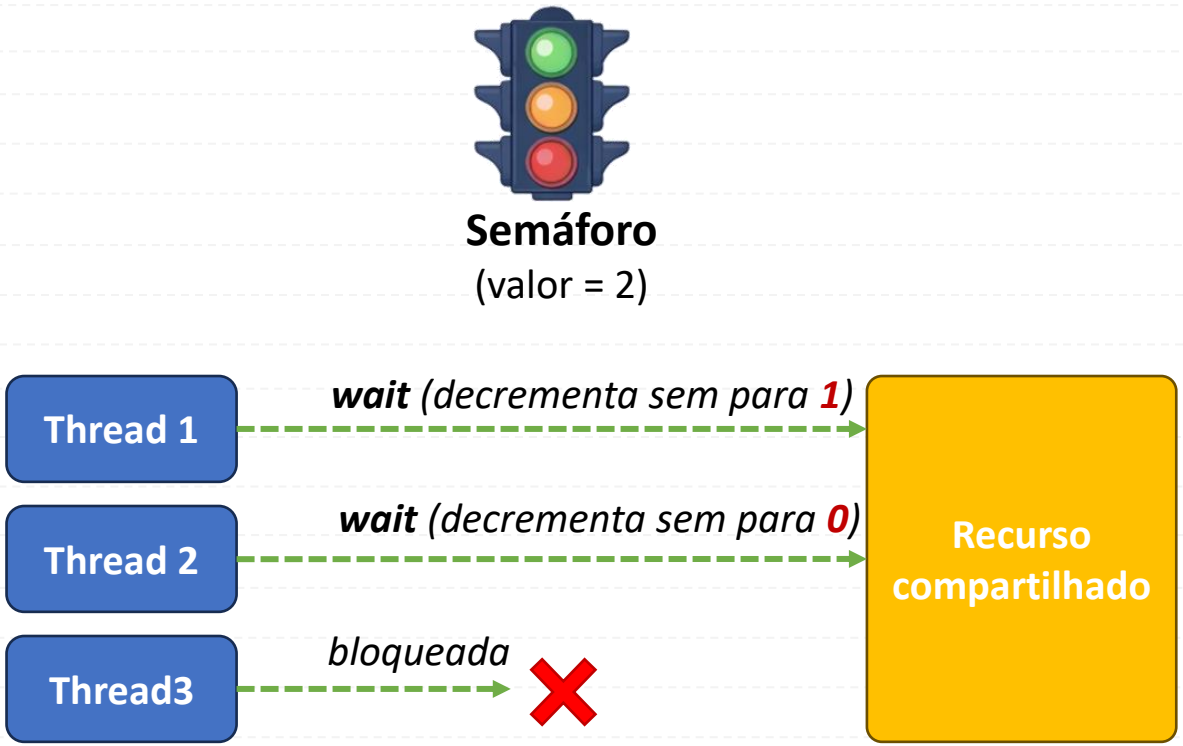
1	Coordenação entre threads e processos.
2	Controle de regiões críticas
3	Comunicação entre processos
2	Limitação entre paralelisto.

Edsger Wybe Dijkstra *foi um* ***cientista da computação holandês****, considerado um dos* ***fundadores da ciência da computação teórica e da programação moderna****. Ele ajudou a transformar programação em uma* ***disciplina científica rigorosa****, propondo algoritmos em grafos, semáforos, sistemas operacionais.*

Semáforo

Semáforo é basicamente uma variável inteira protegida, manipulada por operações atômicas, que controla quantos processos podem acessar um recurso ao mesmo tempo. Suas operações fundamentais são **wait** e **signal**, detalhadas a seguir:





A imagem ilustra um **semáforo contador inicializado com valor 2**, indicando que apenas duas threads podem acessar simultaneamente o recurso compartilhado; assim, a Thread 1 executa a operação ***wait*** e reduz o valor para 1, a Thread 2 faz o mesmo e reduz para 0, ocupando as duas "vagas" disponíveis, enquanto a Thread 3 tenta acessar o recurso, mas como o semáforo já está em 0, ela é **bloqueada** até que alguma das threads em execução libere o recurso por meio da operação ***signal***, que incrementa o valor do semáforo e permite a entrada de outra thread.

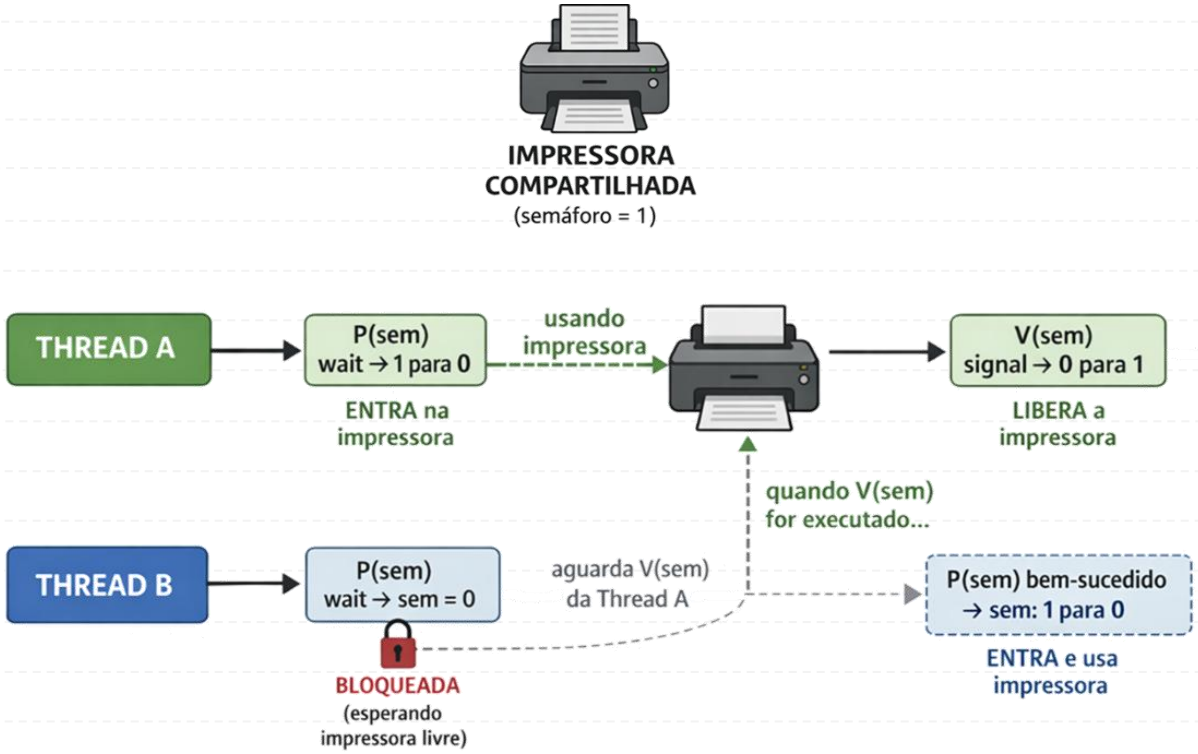
Semáforo

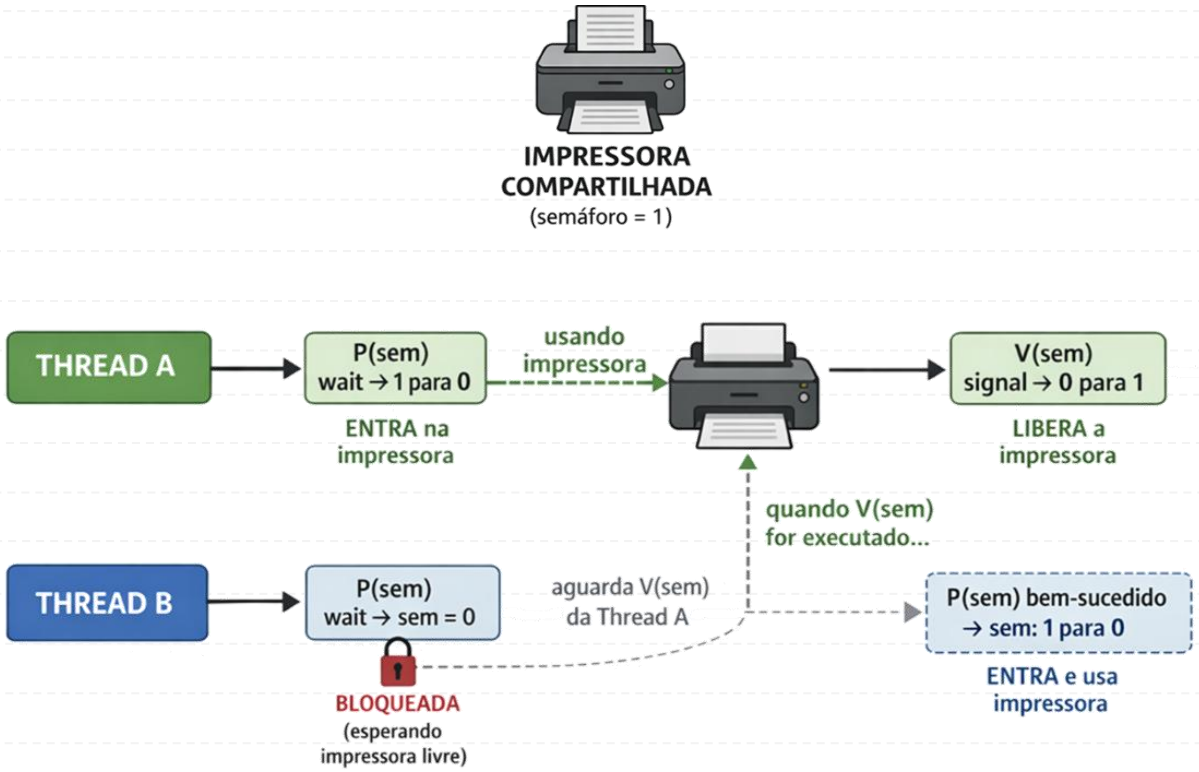
Explique como se dá o desenvolvimento das seguintes instruções, considerando o funcionamento de uma impressora compartilhada em uma rede de computadores, cujo acesso se dá via semáforo:

sem = 1
Thread A:
P(sem)
usa impressora
V(sem)
Thread B:
P(sem)
espera

Semáforo

Explique como se dá o desenvolvimento das seguintes instruções, considerando o funcionamento de uma impressora compartilhada em uma rede de computadores, cujo acesso se dá via semáforo:





*O semáforo foi inicializado com valor **1**, caracterizando um caso de **exclusão mútua** (como um mutex), pois existe apenas **uma impressora compartilhada**. Quando a **Thread A** executa a operação **P(sem)** (wait), o valor do semáforo passa de 1 para 0 e ela entra na seção crítica, podendo usar a impressora. Enquanto isso, a **Thread B** também tenta executar **P(sem)**, mas como o valor já está em 0, ela **fica bloqueada (em espera)** até que a Thread A termine de usar o recurso e execute **V(sem)** (signal), incrementando o semáforo novamente para 1. Somente então a Thread B poderá prosseguir e utilizar a impressora, garantindo que nunca haja uso simultâneo do recurso.*

```python
import threading
import time

sem = threading.Semaphore(2)

def tarefa(id):
    print(f"Thread {id} esperando")
    sem.acquire()
    print(f"Thread {id} entrou")
    time.sleep(2)
    print(f"Thread {id} saiu")
    sem.release()

for i in range(5): threading.Thread(target=tarefa, args=(i,)).start()
```

Semáforo

Um mutex é um tipo de semáforo simplificado, mas útil para a exclusão mútua entre processos leves.

Semáforo

Um mutex é um tipo de semáforo simplificado, mas útil para a exclusão mútua entre processos leves.

Item CORRETO.

Um mutex pode ser visto como uma forma simplificada e especializada de semáforo, normalmente equivalente a um semáforo binário, cujo objetivo principal é garantir exclusão mútua na seção crítica. Ele é amplamente utilizado para sincronizar processos leves (threads), impedindo que mais de uma execute simultaneamente um trecho de código que acessa recurso compartilhado.

Ressalva importante:

*Apesar da semelhança, mutex ≠ semáforo binário em sentido estrito, pois **mutex** possui propriedade de posse (só quem trava pode destravar) e **semáforo** não tem essa restrição. Tecnicamente, a forma mais precisa seria:*

"Mutex é uma primitiva de sincronização equivalente a um semáforo binário, especializada para exclusão mútua entre threads."

```python
import threading

semaphore = threading.Semaphore(3) # Semáforo com limite de 3 threads

def access_resource(thread_id):
    print(f"Thread {thread_id} tentando acessar o recurso...")
    with semaphore:  # Adquire o semáforo
        print(f"Thread {thread_id} acessou o recurso.")
        threading.Event().wait(1)         # Simula uso do recurso
    print(f"Thread {thread_id} liberou o recurso.")

# Criando threads

threads = [threading.Thread(target=access_resource, args=(i,)) for i in range(5)]
for t in threads: t.start()
for t in threads: t.join()
```

O semáforo permite que no máximo 3 threads acessem o recurso simultaneamente (as outras threads aguardam até que uma das threads libere o semáforo.

A situação em que dois ou mais processos ficam permanentemente bloqueados, esperando por recursos que nunca serão liberados, caracteriza um **Deadlock**.

Marcelo Eustáquio

Deadlock

Deadlock (interbloqueio) é uma situação em sistemas concorrentes ou distribuídos em que dois ou mais processos/threads ficam permanentemente bloqueados, cada um esperando por um recurso que está sendo mantido por outro, de modo que nenhum consegue prosseguir. É um problema clássico de sincronização e gerenciamento de recursos.

O modelo clássico é ilustrado a seguir:

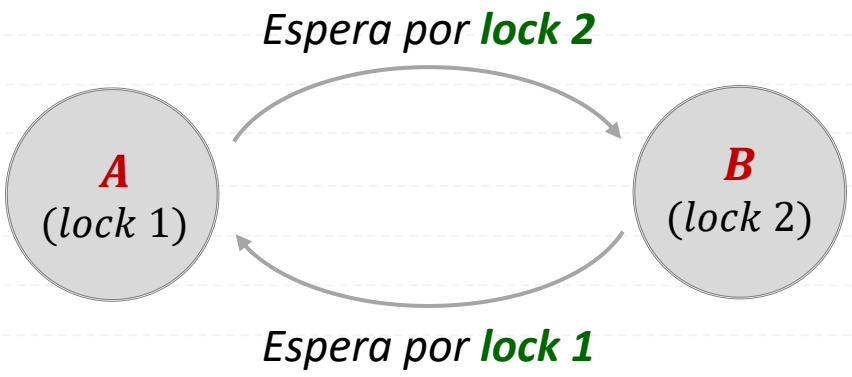
O processo **A** acessa o recurso 1 (*lock 1*) e precisa do recurso 2 (*lock 2*).

O processo **B** acessa o recurso 2 (*lock 2*) e precisa do recurso 1 (*lock 1*).

Tem-se um impasse: A espera por B (recurso de posse de B) e B espera por A (recurso de posse de A).

Deadlock

Deadlock é um caso de impasse: enquanto o processo **A** acessa o recurso 1 (*lock 1*) e precisa do recurso 2 (*lock 2*, o processo **B** acessa o recurso 2 (*lock 2*) e precisa do recurso 1 (*lock 1*).



O diagrama representa um **deadlock clássico por espera circular**. O processo **A** já possui o **lock 1** e passa a **esperar o lock 2**, que está com o processo **B**. Por outro lado, o processo **B** já possui o **lock 2** e passa a **esperar o lock 1**, que está com o processo **A**. Assim, forma-se um ciclo: A → B → A, em que nenhum dos dois consegue avançar nem liberar o recurso que o outro precisa. Como não há preempção (ninguém "toma" o lock do outro) e ambos mantêm seus recursos enquanto aguardam, o sistema entra em **impasse permanente**, caracterizando o deadlock.

Deadlock

Para que um deadlock ocorra, as seguintes condições devem ser verdadeiras (**condições de Coffman**):

1	Exclusão Mútua: Um recurso só pode ser usado por um processo/thread de cada vez.
2	Posse e Espera: Um processo/thread já está segurando um recurso e está esperando por outros.
3	Não Preempção: Recursos não podem ser forçadamente retirados de um processo/thread.
4	Espera Circular: Existe um conjunto de processos/threads em espera circular.

Atenção:

Se qualquer uma das condições falhar, **não há deadlock**.

```python
import threading
import time

lock1 = threading.Lock()
lock2 = threading.Lock()

def tarefa1():
    with lock1:
        print("T1 pegou lock1")
        time.sleep(1)
        print("T1 esperando lock2")
        with lock2:
            print("T1 pegou lock2")
            print("T1 executando região crítica")

def tarefa2():
    with lock1:   # agora também começa por lock1
        print("T2 pegou lock1")
        time.sleep(1)
        print("T2 esperando lock2")
        with lock2:
            print("T2 pegou lock2")
            print("T2 executando região crítica")
```

```python
t1 = threading.Thread(target=tarefa1)
t2 = threading.Thread(target=tarefa2)

t1.start()
t2.start()

t1.join()
t2.join()

print("Fim")
```

Deadlock

Entre as principais estratégias para se evitar deadlocks, recomenda-se:

1	Sempre adquirir locks na mesma ordem *(lock1, lock2, lock3, ...)* para evitar espera circular.
2	Usar timeout, com *lock.acquire(timeout = 2)*, para evitar bloqueio infinito.
3	Evitar múltiplos *locks* simultâneos (quanto menos dependências, menor o risco de deadlock).
4	Usar estruturas de alto nível como filas thread-safe, semáforo, executores.

Exemplos:

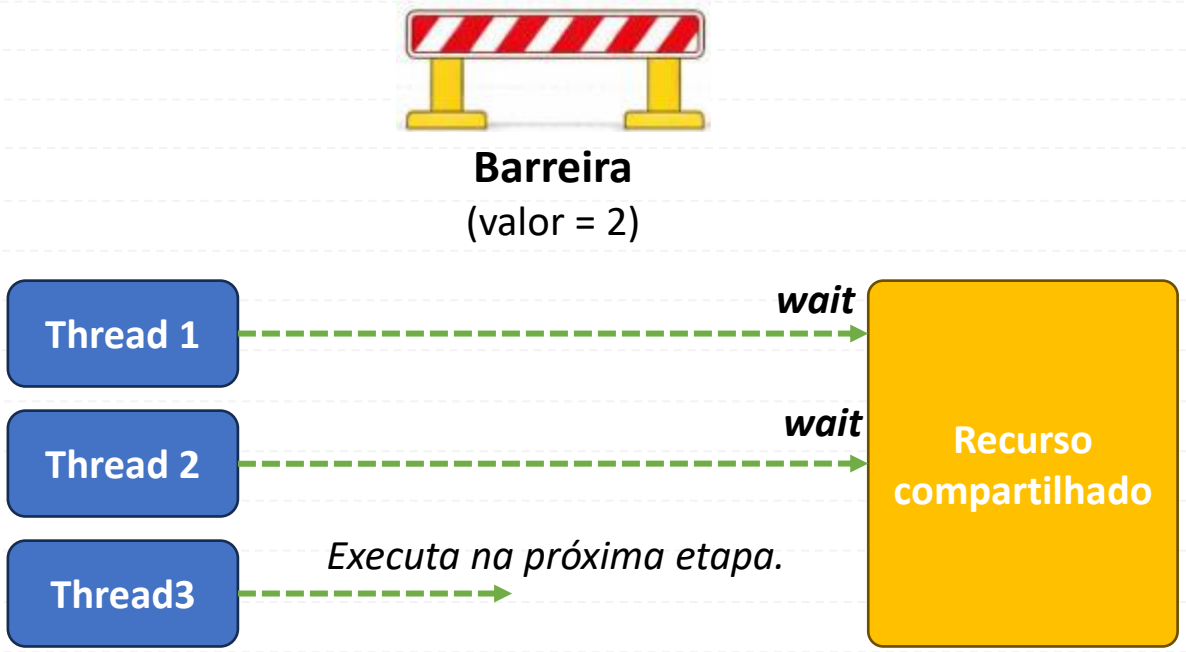
Na prática, deadlock ocorrem em bancos de dados (transações), sistemas operacionais, sistemas distribuídos, microserviços, kernel, sistemas financeiros e sistemas judiciais (filas concorrentes) .

Barreira (barrier) é um mecanismo de sincronização usado para fazer com que um conjunto de threads ou processos espere até que todos atinjam um determinado ponto de execução, antes que qualquer um possa continuar.

Marcelo Eustáquio

Barreiras

Barreira é uma construção que faz com que um grupo de threads espere até que todas tenham alcançado um certo ponto dentro do código-fonte. Em Python, o módulo threading fornece a classe Barrier.



No código a seguir, todas as threads chegam à barreira e só prosseguem quando o número de threads (3, no caso) estiver presente.

```python
import threading

# Barreira para 3 threads
barrier = threading.Barrier(3)

def worker(thread_id):
    print(f"Thread {thread_id} iniciada.")
    threading.Event().wait(1)     # Simula algum trabalho
    print(f"Thread {thread_id} chegou à barreira.")
    barrier.wait()  # Aguarda as outras threads
    print(f"Thread {thread_id} passou pela barreira.")

# Criando threads

threads = [threading.Thread(target=worker, args=(i,)) for i in range(3)]

for t in threads: t.start()
for t in threads: t.join()
```

A situação de Starvation ocorre quando um processo nunca é executado, pois a existência de processos com maior prioridade o impede de entrar em execução.

Marcelo Eustáquio

Starvation

Starvation (ou inanição) é um problema que ocorre em sistemas concorrentes quando um processo ou thread é continuamente ignorado ou impedido de acessar recursos necessários para seu progresso. Isso acontece porque outros processos/threads monopolizam os recursos, deixando o processo "faminto" sem a oportunidade de avançar.

Diferente de um deadlock , onde todos os processos envolvidos ficam paralisados, na starvation apenas alguns processos são prejudicados, enquanto outros continuam funcionando normalmente.

Starvation

Starvation (ou inanição) é um problema que ocorre em sistemas concorrentes quando um processo ou thread é continuamente ignorado ou impedido de acessar recursos necessários para seu progresso. Isso acontece porque outros processos/threads monopolizam os recursos, deixando o processo "faminto" sem a oportunidade de avançar.

São causas do starvation:

1. Prioridade de Processos: processos de baixa prioridade podem nunca ser executados.
2. Na ausência de políticas justas para distribuir recursos, alguns processos podem ser ignorados.
3. Um processo pode ficar esperando indefinidamente por um recurso.
4. Algoritmos como Shortest Job Next (SJN) podem favorecer certos processos e negligenciar outros.

Starvation

Starvation (ou inanição) é um problema que ocorre em sistemas concorrentes quando um processo ou thread é continuamente ignorado ou impedido de acessar recursos necessários para seu progresso. Isso acontece porque outros processos/threads monopolizam os recursos, deixando o processo "faminto" sem a oportunidade de avançar.

Exemplo clássico:

Considere um sistema operacional com três processos:
P1, de alta prioridade; P2, de prioridade média; e P3, de baixa prioridade. Se P1 e P2 estiverem sempre prontos para serem executados, P3 nunca terá a chance de rodar, mesmo que esteja esperando há muito tempo. Esse cenário é um exemplo clássico de starvation.

```python
import threading
import time

# Variáveis globais para rastrear o uso do recurso

high_priority_count = 0
low_priority_count = 0

# Cria um lock para controlar o acesso ao recurso compartilhado

resource_lock = threading.Lock()

# Função executada pela thread de alta prioridade

def high_priority_thread():
    global high_priority_count
    while True:
        with resource_lock:  # Adquire o lock
            print("[ALTA PRIORIDADE] Usando o recurso...")
            high_priority_count += 1
            time.sleep(0.9)  # Simula uso prolongado do recurso
```

```python
# Função executada pela thread de baixa prioridade

def low_priority_thread():
    global low_priority_count
    while True:
        with resource_lock:  # Tenta adquirir o lock
            print("[BAIXA PRIORIDADE] Conseguiu usar o recurso!")
            low_priority_count += 1
            time.sleep(0.1)  # Simula uso rápido do recurso

# Cria as threads

high_priority = threading.Thread(target=high_priority_thread, daemon=True)
low_priority = threading.Thread(target=low_priority_thread, daemon=True)

# Inicia as threads

high_priority.start()
low_priority.start()

# Monitora o uso do recurso por 10 segundos

time.sleep(10)
```

```python
# Relatório final

print("\n--- RELATÓRIO FINAL ---")
print(f"Thread de Alta Prioridade usou o recurso {high_priority_count} vezes.")
print(f"Thread de Baixa Prioridade usou o recurso {low_priority_count} vezes.")
print("------------------------")
```

Programação concorrente

1º semestre de 2025

Sincronização

Tema #05

Professor Marcelo Eustáquio

Q01

Considere:

Thread A:
lock(X)
lock(Y)

Thread B:
lock(Y)
lock(X)

Perguntas:

A) Existe possibilidade de deadlock?
B) Como evitar?
C) Qual condição de Coffman está sendo violada?

Q02

Discorra sobre o Teorema CAP, explicando os conceitos de consistência, disponibilidade e tolerância a partição e por que é impossível garantir os três simultaneamente.

Q03

Considere que **x = 0** e as seguintes threads:

Thread 1: x = x + 1
Thread 2: x = x + 1

Agora responda:

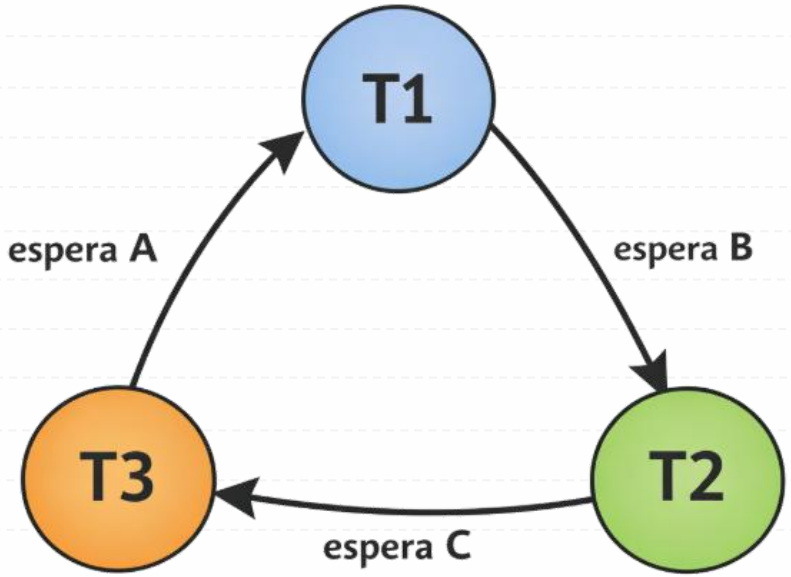
A) Quais resultados possíveis?
B) Modele a interleaving.

Q04

Construa grafo de deadlock para:

I. T1 segura A e espera B
II. T2 segura B e espera C
III. T3 segura C e espera A

Existe deadlock?



Interpretação

•**T1** só continua se conseguir **B**, mas **B** está com **T2**

•**T2** só continua se conseguir **C**, mas **C** está com **T3**

•**T3** só continua se conseguir **A**, mas **A** está com **T1**

Ninguém consegue prosseguir, porque cada thread depende da liberação da próxima.

Conclusão

Há **deadlock**, porque existe **espera circular** no grafo.

Q05

Crie um programa em Python com **duas threads,** em que cada uma deve incrementar uma variável global contador **1.000.000 vezes**. Ao final, o programa deve exibir o valor de contador.

Agora, responda:

1) Qual seria o valor esperado?
2) Por que o valor obtido pode ser diferente?
3) Qual problema de concorrência está ocorrendo?
4) Em seguida, faça uma **segunda versão corrigida**, usando threading.Lock.

Q06

Crie três threads:

- Thread A imprime A
- Thread B imprime B
- Thread C imprime C

O programa deve imprimir a sequência ABC **10 vezes**, sempre nessa ordem.

Exemplo de saída:

ABCABCABCABCABCABCABCABCABCABC

Observação:

Use mecanismos de sincronização da biblioteca threading.

Q07

Crie vários arquivos .txt de teste e desenvolva um código-fonte que procure uma palavra específica em todos eles usando múltiplas threads.

Requisitos:

1) cada thread deve processar um arquivo;
2) ao final, o programa deve informar em quais arquivos a palavra foi encontrada;
3) proteger a estrutura compartilhada de resultados.

Q08

Um programador está desenvolvendo um sistema de controle de estoque no qual múltiplos threads precisam atualizar a quantidade de produtos simultaneamente. Para garantir que a quantidade de produtos seja atualizada corretamente sem causar inconsistências, ele precisa utilizar uma primitiva de sincronização específica.

Considerando-se esse contexto, qual é a propriedade que caracteriza unicamente a primitiva de sincronização chamada ‘mutex’ e que a qualifica para garantir a condição desejada?

Q09

Threads de um mesmo processo compartilham o espaço de endereçamento do processo.

Q10

O mutex é um mecanismo de sincronização de threads o qual permite que mais de uma thread execute um trecho de código simultaneamente.

Q11

Threads criadas dentro de um mesmo processo compartilham o mesmo espaço de endereçamento e recursos, como arquivos abertos, o que facilita a comunicação, mas exige sincronização.

Q12

Considere um sistema de reserva de assentos para um cinema que possui 100 assentos numerados de 1 a 100, e deve permitir que os clientes reservem assentos online. Vários clientes podem acessar o sistema simultaneamente, e cada cliente pode tentar reservar um ou mais assentos por vez.

Sem mecanismos de sincronização, podem ocorrer condições de corrida quando dois ou mais clientes tentam reservar o mesmo assento ao mesmo tempo. Isso pode resultar em problemas como:

1. Reservas duplicadas: Dois clientes reservam o mesmo assento porque o sistema não verifica corretamente se o assento já está ocupado antes de confirmar a reserva.
2. Inconsistência no estado do sistema: O sistema pode exibir informações incorretas sobre a disponibilidade dos assentos, levando a uma experiência frustrante para os clientes.

Para resolver esses problemas, é necessário usar um mutex (trava) para garantir que apenas uma thread (representando uma solicitação de reserva) possa modificar o estado dos assentos por vez.

Q13

Considere um sistema que controle o acesso a uma sala de reuniões em um escritório: a sala tem capacidade máxima para 5 pessoas e funcionários podem entrar e sair da sala ao longo do dia, mas é necessário garantir que:

1. Nenhum funcionário entre na sala se ela já estiver cheia (capacidade máxima atingida).
2. Quando um funcionário sai da sala, ele libera espaço para outros entrarem.

O sistema deve permitir que múltiplos funcionários tentem acessar a sala simultaneamente, mas respeitando a capacidade máxima. Para resolver esse problema, usaremos semáforos, que são mecanismos de sincronização adequados para controlar o acesso a recursos limitados, como a sala de reuniões neste caso.

Q14

Julgue: Diz-se que um ou mais processos estão em starvation quando cada um deles está à espera de recursos que estão ocupados por outros processos, que podem ou não estar no mesmo grupo.

Q15

Julgue: Um dos riscos de uma política de alocação de recursos do tipo primeiro a chegar, primeiro a ser servido é a ocorrência de condição de inanição (starvation) para os processos.

Q16

Julgue: Durante a execução dos processos P1 e P2, se eles não concluírem suas execuções porque o processo P1 depende do término do processo P2 que, por sua vez, depende da conclusão do processo P1, tem-se uma situação denominada starvation.

Q17

Um sistema sempre prioriza threads com menor tempo de execução. Indique problemas que podem surgir ao ser aplicada essa política de escalonamento. Dê um exemplo concreto para ilustrar esse problema.

Q18

Complete o programa para permitir que apenas 2 threads acessem simultaneamente um recurso:

```
import threading
import time

sem = threading.Semaphore(2)

def usa_recurso(nome):
    # entrar
    print(f"{nome} quer entrar")
    # usar recurso
    print(f"{nome} entrou")
    time.sleep(2)
    print(f"{nome} saindo")
    # sair
threads = []
for i in range(5):
    t = threading.Thread(target=usa_recurso, args=(f"Thread-{i}",))
    threads.append(t)
    t.start()
for t in threads: t.join()
```

Q19

Considere o seguinte pseudocódigo:

```
# Pseudocódigo
enquanto houver threads:
    sempre execute a thread de maior prioridade
```

Responda:

1) Que problema pode surgir para threads de baixa prioridade?
2) Por que isso não é necessariamente deadlock?
3) Que técnica poderia reduzir esse problema?

Depois, implemente uma simulação simples em Python usando fila de prioridades ou pseudocódigo comentado.

Q20

Reescreva o código a seguir usando mecanismos de sincronização para sanar eventuais inconsistências que podem ocorrer:

```
import threading

saldo = 100

def sacar(valor):
    global saldo
    if saldo >= valor:
        saldo = saldo - valor

t1 = threading.Thread(target=sacar, args=(80,))
t2 = threading.Thread(target=sacar, args=(80,))
t1.start()
t2.start()
t1.join()
t2.join()

print("Saldo final:", saldo)
```

Q21 Identifique eventuais problemas de concorrência que podem ser observados no código a seguir:

```python
import threading
import time

saldo = 0
lock1 = threading.Lock()
lock2 = threading.Lock()
barreira = threading.Barrier(3)

def tarefa_a():
    global saldo
    with lock1:
        time.sleep(1)
        with lock2:
            saldo += 10
    barreira.wait()
```

```python
def tarefa_b():
    global saldo
    with lock2:
        time.sleep(1)
        with lock1:
            saldo += 20
    barreira.wait()

def tarefa_c():
    global saldo
    saldo += 30
    barreira.wait()

threads = [
    threading.Thread(target=tarefa_a),
    threading.Thread(target=tarefa_b),
    threading.Thread(target=tarefa_c),
]

for t in threads: t.start()
for t in threads: t.join()
print(saldo)
```

Perguntas:

1) Há risco de deadlock?
2) Há condição de corrida?
3) A barreira está sendo usada corretamente?
4) Como você corrigiria o programa?